Dynamo Pay configuration and integration manual

1. Your installation

Use https://dynamo-pay.onrender.com as the gateway base URL. Sign in at /admin; customer documentation is at /developers and /downloads. The demo store is disabled on the hosted gateway. Keep the current forest-green desktop layout, mobile navigation and checkout as the visual baseline.

The hosted launcher runs Payment API, Checkout, Merchant Console and Gmail Observer in one service behind one HTTPS origin. PostgreSQL stores payment and merchant records. SQLite remains the local-development adapter. Redis is optional, not the active source of truth for the hosted financial callback queue. A provider callback and its journal/outgoing callback reservation commit together.

2. Connect a merchant website

Sign in and open Developer tools. Copy Client-ID, Client-Secret and Service-Id into your server's secret manager. Send all three headers and Content-Type: application/json from your backend. Never put Client-Secret or the webhook secret in browser JavaScript. A publishable merchant identifier is not a secret credential.

POST /v1.0/collect/upi creates a direct UPI session. Example body: {"amount":201,"uniquetransactionid":"YOUR-ORDER-1001","remarks":"Order payment","name":"Customer","emailid":"customer@example.com","mobileno":"9000000000"}. Reuse the same order ID when retrying; changing it creates a separate order. The response provides the checkout URL/transaction reference and UPI intent/QR when a receiving slot is available. A queued order has no payable QR until activated. Open the returned checkout URL directly. Do not construct a provider URL yourself.

POST /v1.0/payin/checkstatus accepts {"uniquetransactionid":"YOUR-ORDER-1001"}. GET status variants remain supported where specified by the downloadable OpenAPI contract. POST /v1.0/ifsc/{ifsc} validates an IFSC; POST /v1.0/wallet reads recorded ledger balances. These balances are not an independently queried bank balance. Use /developers/openapi.json for the merchant-facing contract. /api/v1/collections, payouts, banking, refunds, settlement and ledger administration currently require administrator authentication; do not distribute administrator tokens to customers.

Set your webhook URL under Developer tools. It must be a public HTTPS endpoint. Optional IP restrictions accept exact public IPv4/IPv6 addresses, not CIDR ranges. Use the server's outbound IP. A callback URL must accept HTTP POST and acknowledge receipt with a 2xx response after durably recording the event.

3. Confirmations and customer fulfilment

Order creation and the customer's Payment done button do not prove money arrived. In production, SMS, Android notifications and Gmail are supplementary evidence saved for review. Provider or independently authenticated bank confirmation is required for automatic payment success. In Transactions, open Details, choose Review bank credit, enter the 12-digit UTR and a review note, and confirm you checked the exact bank credit. This owner-only action posts the journal and queues the callback atomically. The owner can separately record a manual confirmation after checking the bank; this is labelled MERCHANT, not provider verified. Existing historical records retain their labels. A direct-UPI installation without a bank/provider feed requires this review step.

For a callback, compute HMAC-SHA256 over the exact raw request bytes with your webhook signing secret. Compare against X-Gateway-Signature (sha256=<hex>) using a constant-time comparison. Do not parse and reserialize JSON before checking. Persist X-Gateway-Delivery-Id and order ID with a unique constraint; credit your business wallet once in the same transaction. Check amount, currency, order ownership, event, method and confirmationSource. COD success means order placement; it is not prepaid money. Do not deliver goods or credit a wallet based solely on a browser event.

Callbacks are persisted before delivery, retried up to five attempts and retain the same delivery ID/payload. Your server can receive the same callback more than once. An unreachable endpoint is visible as pending retry or exhausted in Payment operations. Dashboard SSE prompts a current-data refresh, reconnects automatically and falls back to polling every 20 seconds. It provides current-state recovery, not a durable event replay API.

4. Receiving UPI accounts and limits

Open UPI accounts. Save the channel enable/disable setting, minimum amount (normally INR 200), maximum amount (normally INR 20,000) and allowed receiving VPAs. Only a valid allowed receiving VPA can accept a new checkout. Payment acceptance rules on the same page support ALLOW, BLOCK and WATCH for receiving VPAs, submitted customer VPAs, merchant/customer IDs and provider names. Every rule needs a reason; optional expiry, enable/disable and audit history are recorded. BLOCK wins over ALLOW. An ALLOW rule restricts its target category; WATCH records a match without bypassing other checks. These are platform acceptance controls, not a bank or NPCI blacklist. A submitted payer VPA is not independently verified bank identity. A disabled channel rejects new payments. Existing installations must save the channel settings once; unsaved legacy settings retain their previous amount limits.

The current direct-UPI allocator permits one active receiving VPA per merchant and one ACTIVE/PROCESSING payment per account; other payments queue. Stored UPI IDs are not a promise of simultaneous payable QR capacity. Adding IDs does not authorize a higher bank limit. The rate limiter defaults to 1,000 API requests per IP per 60-second fixed window; environment settings can change it. It is per process and resets on restart. HTTP 429 includes Retry-After. This is a request limit, not successful-payment throughput. No verified 1,000-concurrent-paid-checkout or subsecond bank-confirmation claim is made.

5. Connect the receiving Android phone

Use a current signed Dynamo Pay Android observer build from the owner. The repository's old APK predates the hardening changes; do not treat that binary as the updated build. A new native build and real-phone acceptance test are required before rollout. The app is an owner-installed APK, not an app-store product.

On the phone receiving bank SMS, allow installation from your trusted download source. Grant SMS, notification and camera permissions. Enable Notification access for the observer, allow its foreground notification and follow the phone vendor's battery/autostart settings. Keep mobile data or Wi-Fi available.

On the laptop, sign in to Connections & alerts, enter the hosted HTTPS gateway and generate a pairing QR. On your own receiving phone, tap Scan pairing QR. Scan before its ten-minute expiry. Pairing provisions a device ID and signing key; do not share the QR or the key. Never put administrator tokens in the app. SMS event requests and heartbeats use X-Device-Id, X-Timestamp and an HMAC-SHA256 X-Signature over raw JSON + ':' + timestamp. Revoking the device prevents further ingestion.

The updated Java observer source persists pending events before sending and retries during its foreground loop. Original observation time stays separate from the current signing time. Queue capacity is 1,000 events; reconnect and inspect the device before it fills. Test pairing, signed heartbeat, a genuine credit observation, offline/reconnect, phone reboot and duplicate delivery. An SMS/push alert still needs bank/provider

confirmation or owner review before paid status. Arrival speed depends on the bank, phone and network; there is no zero-failure or instant-bank guarantee.

6. Connect Gmail

In Google Cloud, select your project, enable Gmail API and configure the OAuth consent screen. Create a Web application OAuth client. Add exactly https://dynamo-pay.onrender.com/oauth/callback as an authorized redirect URI. For a testing consent screen, add the receiving inbox as a test user. Request only the Gmail read scope used by the observer. Publish/verify the Google application as required for broader use.

In Connections & alerts, save Client ID and Client Secret, then choose Connect Gmail. Sign in to the bank-alert inbox and approve access. Offline access supplies a refresh token to the server. Run Test Gmail and confirm the linked inbox/status. A Google API key alone cannot read a private Gmail inbox. Never paste bank PINs or OTPs into configuration.

Bind GMAIL_RECEIVING_ACCOUNT_ID to the actual receiving-account database ID for this inbox. Do not bind it to whichever customer's order is newest. The current observer is one explicitly bound inbox per service; independent merchant inboxes need independent scoped observer instances before multi-tenant rollout. Credentials are saved for the selected merchant only and encrypted in merchant configuration. The poll interval defaults to 5,000 ms while the host is awake. Gmail polling paginates the previous seven days and deduplicates observations in the backend; a duplicate is not a second payment. Bank email delay and OAuth/network failures add latency. Gmail also remains supplementary evidence.

Troubleshooting: redirect_uri_mismatch means the URI differs in Google Cloud; invalid_grant means reconnect/consent is required; 401 means token refresh/authentication failed; an idle Render service cannot poll. Disconnect clears the selected merchant's Gmail configuration and stops the local observer. Keep refresh tokens private.

7. Refunds, splits and settlements

Refunds in the console record a manual transfer request and reserve refundable money. Send the real bank transfer, then record the actual bank reference. Full and partial requests cannot exceed the remaining balance. Split settings support platform fee/GST and recipient percentage allocations; recording a transfer does not itself send money.

Provider refunds reserve the amount before the provider request. Reuse refundId on retry. A timeout may mean the provider already accepted the refund; RECONCILIATION_REQUIRED retains the reservation until checked. Do not create another refund ID just because a response was lost. Automatic settlement endpoints remain unavailable until a bank/provider settlement integration is configured; they never invent a bank reference. Direct bank host-to-host integration requires bank agreements/certificates and is not activated by entering a UPI ID. Card and netbanking payments require an approved hosted provider checkout and are unavailable through the legacy local simulator in production.

8. Hosting, backup and release procedure

Set DATABASE_URL privately to the managed Postgres connection string. Preserve ENCRYPTION_MASTER_KEY, JWT_SECRET and existing internal/merchant secrets. npm ci --include=dev, npm run typecheck, npm run build and npm test are the normal checks; start with npm run start:hosted. Use /_ready for database readiness. The SQLite-to-Postgres migration refuses an occupied target and preserves a local backup. Do not run data-reset scripts on production.

Render Free sleeps when idle; in-process Gmail polling stops while asleep and wake-up is not immediate. The current free database expires on 30 November 2026. Export a private consistent backup and arrange durable hosting before expiry; no infrastructure upgrade is automatic. Protect backups as credentials and financial data. Restore to a separate database first and verify counts, balances, private-file access and keys before changing DATABASE_URL.

Freeze procedure: record the accepted commit SHA, versions, schema and PDF hashes; preserve database and secrets backups; pass regression, isolated PostgreSQL, browser and device checks; review external blockers; create an annotated release tag. Change only on a review branch after freeze. A tag is a fixed source baseline, not an uptime guarantee. Production deployment and credential rotation remain separate authorized actions.